TAREA M30 – CD-YAIR PALOMARES

```
import numpy as np
import pandas as ps
import matplotlib.pyplot as plt
from sklearn.neighbors import NearestNeighbors
```

- Problema 1: Extraiga del internet la base de datos "wine-clustering.csv" de la siguiente liga y agréguela a un DataFrame en Python. https://www.kaggle.com/datasets/harrywang/wine-dataset-for-clustering

```
df = pd.read_csv("wine-clustering.csv")
df
```

	Alcohol	Malic_Acid	Ash	Ash_Alcanity	Magnesium	Total_Phenols	Flavanoids	Nonflavanoid_Phenols	Proanthocyanins	Color_Intensity	Hue	OD280	Proline
0	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050
2	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185
3	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480
4	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735
...	...	...	...	...	...	...	...	...	...	...	...	...	...
173	13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740
174	13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750
175	13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835
176	13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840
177	14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560

178 rows × 13 columns

- Mediante el algoritmo de K vecinos más cercanos determine el valor promedio de alcohol que tendrían los 5 vinos más parecidos a aquel con las características siguientes:

```python
observacion = {'Alcohol':[14], 'Malic_Acid':[2], 'Ash':[2.5], 'Ash_Alcanity':[16], 'Magnesium':[115],
       'Total_Phenols':[3], 'Flavanoids':[2.5], 'Nonflavanoid_Phenols':[0.4],
       'Proanthocyanins':[2], 'Color_Intensity':[9], 'Hue':[1], 'OD280':[3.5], 'Proline':[800]}
observacion = pd.DataFrame(observacion)
observacion
```

	Alcohol	Malic_Acid	Ash	Ash_Alcanity	Magnesium	Total_Phenols	Flavanoids	Nonflavanoid_Phenols	Proanthocyanins	Color_Intensity	Hue	OD280	Proline
0	14	2	2.5	16	115	3	2.5	0.4	2	9	1	3.5	800

```python
df_array = np.array(df)
observacion_array = np.array(observacion)
```

```python
#Ejecución del algoritmo KNN
k = 5
knn_model = NearestNeighbors(n_neighbors = k, algorithm="auto").fit(df_array)
distances, indices = knn_model.kneighbors(observacion_array)
```

```python
print("K vecinos más cercanos:")
for rank, index in enumerate(indices[0][:k], start=1):
    print(str(rank) + " es", df_array[index])
```

```
K vecinos más cercanos:
1 es [1.356e+01 1.710e+00 2.310e+00 1.620e+01 1.170e+02 3.150e+00 3.290e+00
 3.400e-01 2.340e+00 6.130e+00 9.500e-01 3.380e+00 7.950e+02]
2 es [1.406e+01 1.630e+00 2.280e+00 1.600e+01 1.260e+02 3.000e+00 3.170e+00
 2.400e-01 2.100e+00 5.650e+00 1.090e+00 3.710e+00 7.800e+02]
3 es [1.305e+01 2.050e+00 3.220e+00 2.500e+01 1.240e+02 2.630e+00 2.680e+00
 4.700e-01 1.920e+00 3.580e+00 1.130e+00 3.200e+00 8.300e+02]
4 es [1.293e+01 3.800e+00 2.650e+00 1.860e+01 1.020e+02 2.410e+00 2.410e+00
 2.500e-01 1.980e+00 4.500e+00 1.030e+00 3.520e+00 7.700e+02]
5 es [1.336e+01 2.560e+00 2.350e+00 2.000e+01 8.900e+01 1.400e+00 5.000e-01
 3.700e-01 6.400e-01 5.600e+00 7.000e-01 2.470e+00 7.800e+02]
```

```
df_indices = df.iloc[indices[0]]
df_indices
```

	Alcohol	Malic_Acid	Ash	Ash_Alcanity	Magnesium	Total_Phenols	Flavanoids	Nonflavanoid_Phenols	Proanthocyanins	Color_Intensity	Hue	OD280	Proline
40	13.56	1.71	2.31	16.2	117	3.15	3.29	0.34	2.34	6.13	0.95	3.38	795
20	14.06	1.63	2.28	16.0	126	3.00	3.17	0.24	2.10	5.65	1.09	3.71	780
25	13.05	2.05	3.22	25.0	124	2.63	2.68	0.47	1.92	3.58	1.13	3.20	830
21	12.93	3.80	2.65	18.6	102	2.41	2.41	0.25	1.98	4.50	1.03	3.52	770
141	13.36	2.56	2.35	20.0	89	1.40	0.50	0.37	0.64	5.60	0.70	2.47	780

```
promedio_alcohol = df_indices["Alcohol"].mean()
promedio_alcohol
```

```
13.392000000000001
```

Asegúrese de detallar las concentraciones de alcohol de cada elemento que haya sido utilizado para obtener dicho valor promedio. Problema 2: Considere la lista de 11 compras desglosada de la manera siguiente:

```
my_basket=[['bread','butter','wine','bananas','coffee','carrots'],
['tomatoes','onions','cheese','milk','potatoes'],
```

```
# Lista de 11 compras
my_basket = [['bread', 'butter', 'wine', 'bananas', 'coffee', 'carrots'],
             ['tomatoes', 'onions', 'cheese', 'milk', 'potatoes'],
             ['beer', 'chips', 'asparagus', 'salsa', 'milk', 'apples'],
             ['olive oil', 'bread', 'butter', 'tomatoes', 'steak', 'carrots'],
             ['tomatoes', 'onions', 'chips', 'wine', 'ketchup', 'orange juice'],
             ['bread', 'butter', 'beer', 'chips', 'milk'],
             ['butter', 'tomatoes', 'carrots', 'coffee', 'sugar'],
             ['tomatoes', 'onions', 'cheese', 'milk', 'potatoes'],
             ['bread', 'butter', 'ketchup', 'coffee', 'chicken wings'],
             ['butter', 'beer', 'chips', 'asparagus', 'apples'],
             ['tomatoes', 'onion', 'beer', 'chips', 'milk', 'coffee']]
```

- Modifica el código explicado en este módulo para que sea capaz de realizar un análisis mediante el algoritmo "Market basket" a dicha lista de compras con la finalidad de detectar patrones de consumo. Realice a continuación las conclusiones que sean pertinentes sobre sus resultados.

```
# Ciclo para obtener los productos únicos
productos_unicos = []
for i in range(len(my_basket)):
    for j in range(len(my_basket[i])):
        if my_basket[i][j] not in productos_unicos:
            productos_unicos.append(my_basket[i][j])

# Ciclo para obtener nombres de compras
index = []
for i in range(len(my_basket)):
    index.append("Compra No." + str(i+1))

# Creación de dataframe con ceros
df_my_basket = pd.DataFrame(columns = productos_unicos, index=index).fillna(0)

# Se rellena con 1 la información que corresponde
for i in range(len(my_basket)):
    for j in my_basket[i]:
        df_my_basket[j][i] = 1
df_my_basket
```

	bread	butter	wine	bananas	coffee	carrots	tomatoes	onions	cheese	milk	...	asparagus	salsa	apples	olive oil	steak	ketchup	orange juice	sugar	chicke wing
Compra No.1	1	1	1	1	1	1	0	0	0	0	...	0	0	0	0	0	0	0	0	
Compra No.2	0	0	0	0	0	0	1	1	1	1	...	0	0	0	0	0	0	0	0	
Compra No.3	0	0	0	0	0	0	0	0	0	1	...	1	1	1	0	0	0	0	0	
Compra No.4	1	1	0	0	0	1	1	0	0	0	...	0	0	0	1	1	0	0	0	
Compra No.5	0	0	1	0	0	0	1	1	0	0	...	0	0	0	0	0	1	1	0	

```python
# Aplicación del algoritmo "a priori" para localizar los artículos frecuentemente comprados en la base de datos
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
```

```python
# Análisis "a priori" con un soporte mínimo del 3%
articulos_frecuentes = apriori(df_my_basket, min_support=0.03,
                               use_colnames = True).sort_values("support", ascending=False).reset_index(drop=True)
articulos_frecuentes["length"] = articulos_frecuentes["itemsets"].apply(lambda x:len(x))
articulos_frecuentes
```

```
C:\Users\yair_\AppData\Roaming\Python\Python311\site-packages\mlxtend\frequent_patterns\fpcommon.py:161: DeprecationWarning: DataFrames with non-bool ty
pes result in worse computationalperformance and their support might be discontinued in the future.Please use a DataFrame with bool type
  warnings.warn(
```

	support	itemsets	length
0	0.545455	(butter)	1
1	0.545455	(tomatoes)	1
2	0.454545	(milk)	1
3	0.454545	(chips)	1
4	0.363636	(bread)	1
...	...	...	...
391	0.090909	(carrots, olive oil, butter)	3
392	0.090909	(coffee, butter, chicken wings)	3
393	0.090909	(coffee, butter, sugar)	3
394	0.090909	(coffee, butter, ketchup)	3
395	0.090909	(apples, milk, beer, asparagus, salsa, chips)	6

396 rows × 3 columns

```python
# Obtención de patrones entre productos
association_rules(articulos_frecuentes, metric = "confidence",
                  min_threshold = 0.9).sort_values("confidence", ascending = False).reset_index(drop=True)
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	k
0	(beer)	(chips)	0.363636	0.454545	0.363636	1.0	2.200000	1.0	0.198347	inf	0.857143	0.800000	1.0	
1	(ketchup, onions, wine)	(orange juice)	0.090909	0.090909	0.090909	1.0	11.000000	1.0	0.082645	inf	1.000000	1.000000	1.0	
2	(orange juice, wine)	(ketchup, chips)	0.090909	0.090909	0.090909	1.0	11.000000	1.0	0.082645	inf	1.000000	1.000000	1.0	
3	(ketchup, wine)	(orange juice, chips)	0.090909	0.090909	0.090909	1.0	11.000000	1.0	0.082645	inf	1.000000	1.000000	1.0	
4	(orange juice)	(wine, ketchup, chips)	0.090909	0.090909	0.090909	1.0	11.000000	1.0	0.082645	inf	1.000000	1.000000	1.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
2201	(carrots, butter, wine)	(bread)	0.090909	0.363636	0.090909	1.0	2.750000	1.0	0.057851	inf	0.700000	0.250000	1.0	
2202	(bananas)	(coffee, butter, bread)	0.090909	0.181818	0.090909	1.0	5.500000	1.0	0.074380	inf	0.900000	0.500000	1.0	
2203	(bread, bananas)	(coffee, butter)	0.090909	0.272727	0.090909	1.0	3.666667	1.0	0.066116	inf	0.800000	0.333333	1.0	
2204	(butter, bananas)	(coffee, bread)	0.090909	0.181818	0.090909	1.0	5.500000	1.0	0.074380	inf	0.900000	0.500000	1.0	
2205	(salsa)	(apples, milk, beer, asparagus, chips)	0.090909	0.090909	0.090909	1.0	11.000000	1.0	0.082645	inf	1.000000	1.000000	1.0	

2206 rows × 14 columns

```python
# Conclusiones
print("Conclusiones:")
print("Luego del análisis, se lograron obtener un total de 2,206 relaciones entre compras con una confianza del 100%, por lo que hay demasiada informaci
```

```
Conclusiones:
Luego del análisis, se lograron obtener un total de 2,206 relaciones entre compras con una confianza del 100%, por lo que hay demasiada información para
realizar estrategias de marketing y de acomodo de los productos en el supermercado.
```